SANKUR KUNDU

Curriculum Vitae | ML Systems Engineer & Independent Researcher

Bhubaneswar, India (open to relocation) | +91 97761 94201 | sankur.kundu.tw@gmail.com

linkedin.com/in/sankur-kundu | github.com/Shakhtar-Sankur | shakhtar-sankur.github.io

SUMMARY

ML systems engineer and independent researcher (B.Tech CSE, 2026), sole author of three manuscripts, and co-founder of Gigzen, with pull requests at **vLLM**, **PyTorch** and **Meta ExecuTorch** (the ExecuTorch one approved by two maintainers). I build systems from scratch and measure them against the best in the field: GPU kernels at **cuBLAS parity**, an ML compiler faster on CPU than **torch.compile, ONNX Runtime and JAX/XLA**, an LLM engine at **1.05×** vLLM's throughput with identical output, an open early-warning system for scams across **49** languages, and a tokenizer that beats **GPT-4o's and Gemini's** in 23 of 24 Indian languages. In use today: Populace has **1,600+ npm downloads** (400+ in the last month), and kessler publishes a screening of every active satellite each day. Every number below is reproducible from scripts in a public repository.

MANUSCRIPTS AND RELEASES

- S. Kundu. "Removing the Token Tax from an Existing Model: A Vocabulary Extension for 24 Indian Languages That Keeps Every Other Token." Manuscript, 2026; arXiv (cs.CL) submission in progress.
- S. Kundu. "The Sampler Was Not the Policy: Finding and Removing a Silent Training–Inference Mismatch in a From-Scratch GRPO Stack." Technical report, 2026; arXiv submission in progress.
- S. Kundu. "Scam Defence Stops at English: Measuring the Multilingual Protection Gap and Linking Scam Campaigns Across Languages." Manuscript in preparation, 2026.
- Released **lipi-Indic 96K** on **Hugging Face** (Apache-2.0, October 2026): a tokenizer for 24 Indian languages that keeps every English and code token id of Qwen 2.5.
- Published a study page for each of 11 projects at **shakhtar-sankur.github.io**: method, results, stated limitations and citation, with code and raw results for every number.

OPEN SOURCE CONTRIBUTIONS

- **vLLM #60955** (under review; labelled bug and nvidia): running vLLM's FlashInfer tests on a T4 against a float64 reference showed the prefill tests compared non-causal attention with a causal reference and passed only on a loose tolerance; with the fix, **32 of 32** cases pass and **all 32** fail if the bug returns.
- **Meta ExecuTorch #23677** (approved by two maintainers): the 2-bit and 4-bit quantized embedding kernels used by on-device LLMs read memory out of bounds on an invalid token id; added the bounds check the 8-bit kernel has, with tests that fail without it (quantized kernel suite **82 of 82**).
- **PyTorch #199875** (under review): torch.compile now fills torch.empty in deterministic mode as eager does, without filling Inductor's own buffers, which is what got an earlier fix reverted.
- **PyTorch #199441** and **#199646** (under review): checked all 37 Inductor-skipped tests in test_torch.py on CPU and two T4s, removing **13** stale skips; traced 19 skipped determinism tests to a mode mismatch, so **8** now run under torch.compile.

RECOGNITION

- **Entrepreneur First** (UK): approached unprompted in October 2026 by a Talent Investor who had reviewed tessel's GEMM results (cuBLAS parity), switchyard's MoE serving and a published negative result (a compression pipeline that aimed for 8× smaller and measured −6.3%), and invited to discuss my work.

PUBLIC-INTEREST AI AND SPACE SAFETY

babelguard — Python: an open early-warning system for scams that cross languages (in progress) · study page

- Measured the largest open set of reported scam SMS (**33,869** in **49** languages): South Asian languages are **0.8%** of it and African **0.1%**; **96** scam templates recur across languages, and genuine messages exist for only **4 of 49** languages.
- Measured the protection gap: an English keyword filter catches **86%** of English scams but **10–36%** of Korean, Japanese, Urdu, Arabic, Thai and Hindi ones, and Bengali scams **38%** of the time in Bengali script against **77%** in Latin letters.
- Built linking by meaning that recognises a translated scam **85%** of the time (text matching: **1.5%**) and a crash-safe streaming engine: **4,057 messages/s** on 4 cores, **0.21 KB** per message at 1M, nothing lost in **50** kills.

lipi — Python and PyTorch: cutting the token tax on Indian languages · study page

- Measured the token tax of 24 Indian languages on 15 tokenizers (Odia: **4.99×** English on GPT-4o); built **lipi-Indic** (**on Hugging Face**): fewer tokens than **GPT-4o's and Gemini's** in **23 of 24** languages (**1.30×** vs 1.92×, 1.58×).
- Its 96K new merges fire only on Indian-script bytes, so English, code and other scripts keep exactly their Qwen 2.5 token ids; Santali, the costliest language measured (**13.7×** English on GPT-4o), drops to **1.52×**.
- Extended Qwen2.5-0.5B's vocabulary and continued pre-training on 2× T4 (DDP, ZeRO-1): Odia takes **8.6× fewer** tokens, and in equal GPU time the model predicts held-out Odia **20% better**, with Odia→English chrF++ 16.0 → 24.8.

kessler — C++/CUDA and Python: an independent auditor of space traffic · live globe

- Built SGP4 and all-vs-all conjunction screening in C++/CUDA: all **32,593** tracked objects, every pair, a day ahead in **28 s** on a T4; matches CelesTrak SOCRATES to **0.24 m** (median) and NASA CARA to **1.6e-8**.

- Runs in public every day: a GitHub Actions pipeline screens every active satellite from CelesTrak's open catalogue and publishes a 3D globe; on the GPU, SGP4 computes **125.7 million** orbit states a second in fp64, within **1e-10 km** of Vallado's reference.
- Built a maneuver detector, with debris (which cannot maneuver) as the control group, to audit satellite operators' collision-avoidance reports against public orbit history.

ML SYSTEMS: TRAINING, INFERENCE AND RL

ratchet — Python, PyTorch and C++/CUDA: reinforcement-learning post-training (GRPO) from scratch · study page

- Built GRPO post-training end to end: rollouts with per-token log-probabilities on relay (through a C API), a PyTorch trainer data-parallel on tandem, in-place GPU weight sync, partial rollouts and one-step-ahead training.
- On 2× T4, 100 steps lifted Qwen2.5-0.5B on GSM8K from **48.1% to 52.5%** (all 1,319 test problems); weight sync is **369×** faster than a reload, and overlapping generation with training makes steps **1.9×** faster.
- Found that the engine's fp16 weights rounded away most updates, so it sampled a policy the trainer never had (gap up to 5.2 nats); mixed-precision rounding cut it to **0.04**, and taking the gap apart showed sync rounding outweighs fp16 arithmetic **9–22×**.

relay — C++, CUDA and Swift: a disaggregated, fault-tolerant LLM inference cluster · study page

- Built C++/CUDA prefill and decode workers on custom flash-attention and flash-decoding kernels, a KV-cache transfer engine that streams each layer while the next computes, and a Swift control plane with prefix-aware routing.
- On a T4, cut 2,048-token prefill **8.7×** (3.7 s → 0.43 s) and batch-32 decode **5.4×**; streaming hides **99–100%** of the KV transfer. Output matches Hugging Face and stays bit-identical when batched, cached or split across workers.
- A gather kernel packs each layer's fp16 KV pages into pinned memory on a separate copy stream, ordered by per-layer CUDA events: a 46 MB transfer that cost 21.6 ms after prefill costs **0.08 ms** when streamed.
- Fault-tolerant: watchdogs catch killed, frozen or stalled workers and their requests resume elsewhere with identical output (**384/384** through 60 kills; no request lost in **300** fault-injection seeds).

switchyard — CUDA, C++ and Python: mixture-of-experts serving with expert parallelism · study page

- Served OLMoE-1B-7B (64 experts, top-8) across 2× T4 with expert parallelism built from scratch: CUDA kernels for routing, sorting and grouped expert GEMM/GEMV on tensor cores, and peer-to-peer dispatch in place of NCCL.
- Decodes **2.2× faster** than Hugging Face (112 vs 52 tokens/s at batch 8), bit-identical to one GPU; a decode MoE layer runs **5.4–7×** faster than a per-expert loop, and dispatch is **1.25×** faster than NCCL all-to-all at 256 tokens/GPU.
- Measured on real text: dropping tokens at full experts, as capacity-limited routing does, raises perplexity from **12.9 to 20.2** even at capacity factor 2, so the kernels never drop one. Found and fixed two cross-stream races between the GPUs.

crucible — Python on Linux: sandboxed environments for RL on coding agents · study page

- Built sandboxes from Linux namespaces, seccomp-BPF and limits; a live one forks in **4.5 ms** vs 138 ms to rebuild. Its OpenEnv-compatible grader passes **0 of 9** reward hacks (the usual grader: 4).
- GRPO on ratchet with every reward from a sandbox: MBPP pass@1 **34.9% → 38.6%** (3 seeds; paired test, **p = 0.013**).

tandem — Python, C++ and CUDA: distributed training from scratch, without torch.distributed or NCCL · study page

- Built DDP, ZeRO-1/2/3 (FSDP-style sharding), GPipe and 1F1B on a custom ring all-reduce; a C++/CUDA engine (lock-free shared-memory rings, CUDA IPC, a fused reduce kernel) runs all-reduce **1.6–2.8×** faster than the Python one.
- On 2× T4, DDP and ZeRO-1/2/3 train **bit-identically** to PyTorch DDP/FSDP on NCCL, with all-reduce within **2%** of NCCL's bandwidth at 64–256 MB; on an 86M-parameter GPT, ZeRO-3 cuts per-GPU memory 17%.

COMPILERS AND GPU KERNELS

tessel — Rust, zero dependencies: a tile language and compiler for GPU and TPU kernels · study page

- Designed a Triton-style tile language and wrote its compiler: typed tile IR, linear layouts, CUDA for tensor cores (ldmatrix, mma.sync, swizzled shared memory, software pipelining), an auto-tuner, and Metal and TPU (Pallas) backends.
- On a T4, checked against fp64 references: GEMM at **parity with cuBLAS** (1.00–1.04×), causal attention **1.3–1.9×** faster than PyTorch SDPA, RMSNorm 1.27× faster than torch.compile, softmax 1.07× faster than Triton.
- Built an LLM engine on its kernels alone (paged KV cache, continuous batching, CUDA graphs): on TinyLlama-1.1B its output matches **vLLM** token for token at **1.05×** vLLM's throughput with 32 sequences.

kiln — Rust, zero dependencies: an ML compiler from ONNX to SIMD C and CUDA · study page

- Reads ONNX, optimizes and fuses the graph, generates and compiles SIMD C and CUDA (NVRTC) code, and auto-tunes matrix multiplies on the target machine; its output matches PyTorch's.
- On 4 CPU cores, the lowest median latency on all three benchmark models: BERT in **117 ms** against 149 for torch.compile, 153 for ONNX Runtime and 189 for JAX/XLA; on a T4, SmolLM2 **1.65×** faster than torch.compile.

ferrolm — Rust, zero dependencies: an LLM inference server and RAG stack

- Paged KV cache, continuous batching, prefix caching and speculative decoding on hand-written AVX-512/AVX2 kernels; output matches Hugging Face: **3.4×** the throughput of one-at-a-time serving, first token **46×** sooner with prefix caching; retrieval lifts a 1.7B model's SQuAD F1 from **15 to 48**.

DISTRIBUTED SYSTEMS AND VERIFICATION

quorumdb — Rust, zero dependencies: a distributed SQL database, PostgreSQL wire-compatible · study page

- Built a distributed SQL database that psql connects to: LSM storage, Raft and self-rebalancing multi-Raft, serializable ACID transactions (Percolator) and a SQL planner; **1,500 tpmC** on a TPC-C-derived benchmark.
- Verified by deterministic simulation (**10,000** Raft clusters, 610K crashes, histories machine-checked) and a TLA+ model (337K states): found **6 real bugs** and catches all 15 planted ones.

faultline — Rust, zero dependencies: Jepsen-style fault injection for etcd

- Runs a real cluster in network namespaces under partitions, kill -9 and SIGSTOP, and checks histories for linearizability: **1.26M** operations under **288** faults across 4 etcd release lines; traced lost updates under etcd locks to a paused leader applying a timed-out write **6 s** late; fenced writes lost none.

IN PROGRESS

- **vani** (speech recognition for Indian languages): OpenAI Whisper has no Odia language token and spends **12.7×** English's tokens on Odia text (FLORES, 1,012 sentences); next, Whisper, Meta and Google on FLEURS.
- **Orbital Watch**: every detectable orbit change of every satellite from open element sets, tagged with geomagnetic storm conditions (NOAA Kp) so storm drag is not mistaken for a maneuver.
- **arbiter** (battery bidding in Texas's ERCOT market): the perfect-hindsight ceiling as a linear program over energy and five ancillary services, checked against dynamic programming, two solvers and an independent replay.

EXPERIENCE

Gigzen — Co-founder & Founding Engineer **Apr 2026 – Present**

Early-stage startup (incorporation in progress) · Sole engineer across every product

- Sole engineer on **Waggle**, a delivery platform of three Android apps (riders, shops, customers) from one TypeScript/React/Capacitor codebase on Supabase PostgreSQL: **53 tables, 90 RLS policies, 154 SQL functions**.
- Put first-accept-wins dispatch, UPI reconciliation and GST billing in the database, where no client can bypass them.
- Security-tested it with **150 automated attacks** on its database access rules (**all blocked**) and a 42-step Playwright run.
- Built **Populace**, an open-source tool that fills an app with simulated users acting through its real API: published on **npm** (**1,600+ downloads** in 2026, 400+ in the last month), with a GitHub Action and a Windows app; its first run found **5 defects in 3.5 minutes** after manual testing had passed, and local runs have made **3.17 million** API calls in a day with zero API failures.

Freelance Data Scientist — Upwork, remote (client in Spain) **Jun 2026 – Jul 2026**

- Merged two hotel censuses (5,909 and 8,058 records) and built a Google Trends panel of **10,038 city-months** in Pandas.

EARLIER PROJECTS

Public, each with a README that states its target and what it actually measures, and CI on every push: an **ML serving platform** (versioned rollout, A/B routing, circuit breaking, drift detection); **Detek** (data-leak detection on AWS, BERT plus LSTM); **predictive maintenance** on GCP (LSTM, 48-hour horizon); **edge model compression** (distillation, pruning, int8 to ONNX); **synthetic data with diffusion**; **multi-modal code intelligence**; **software-defined radio**.

SKILLS

GPU and compilers: CUDA (tensor cores, mma.sync, ldmatrix, streams, peer-to-peer, graphs), NVRTC, Triton-style tile languages, Metal, Pallas (TPU), ONNX, SIMD (AVX-512/AVX2)
ML systems: GRPO/PPO post-training, rollout-trainer weight sync, DDP, ZeRO-1/2/3, FSDP, 1F1B, MoE expert parallelism, disaggregated serving, paged KV cache, speculative decoding, AWQ, BPE tokenizers
Distributed systems and verification: Raft, multi-Raft, MVCC, Percolator, LSM trees, deterministic simulation, TLA+, linearizability checking, fault injection
Languages and platforms: C++20, CUDA C++, Rust, Python, Swift (C++ interop), TypeScript, SQL (PL/pgSQL); PyTorch, Hugging Face, PostgreSQL, Supabase, Docker, Kubernetes, GitHub Actions, GCP, AWS

EDUCATION

B.Tech, Computer Science and Engineering (AI & ML) — IEM Kolkata **Sep 2022 – Apr 2026**